Ex 2.2.10: three reads before the handover re-run

> Ex-2.2.9's three loose ends, read off its stored runs: the removal miss on the order-sensitive ops is the line-picking rule's, the retention drop happens before the anneal, and the op1 alignment rises under either half of the handover. We propose the re-run.

Ex-2.2.9 moved the anchor onto the larger grammar and left three loose ends. Removal was one-sided on the three order-sensitive ops, one seed in twenty fell under the retention gate, and the non-red alignment at op1 sat above the old reference. This notebook reads each off the stored runs, with one small scoring pass for the cube figures and no new training.

The removal miss comes from the rule we used to pick the lines. The projection acts like a change in the hue of red, and the lines that missed are the ones whose answer takes only the saturation or value of the red operand. Red is the one anchored concept, so these reads cannot say whether another concept would come apart the same way.

The retention drop happens before the anneal begins. The op1 alignment rises under either half of the handover, so it belongs to the new grammar rather than to the readout alone.

Observations

None of the lines below is a result; ex-2.2.11 will adopt what it needs from here and score it at fresh seeds.

- Removal: on the three order-sensitive ops (`hue-hsv`, `sat-hsv`, `value-hsv`; `hsvmix` is commutative and cleared the gate in ex-2.2.9) the kept share follows a change of hue on the red operand, and not the to-zero rule we used to pick the removal lines. Where the answer takes only the saturation or value of the red operand (`sat-hsv` and `value-hsv` with red at op2), two thirds of the clean exact match survives the projection. Where the answer takes the hue of red, or the whole color, it goes. One case no counterfactual predicts: `hue-hsv` with red at op1, whose answer needs only the saturation and value of red, and where a third of the clean exact match survives; its projected answers keep the hue of op2 and come out paler. A linear probe on the stream reads the projected red operand as a hue rotated away from red, at a lower value, and the rotation grows block by block.
- Retention: on `handover` the alignment reaches a noisy plateau by epoch 10, and its peak is just the high point of that noise. By the time the anneal begins at epoch 45 the seeds sit at 0.66, a few of them drifting downward. Through the anneal itself every condition is flat, so the gate is reading the noise in the plateau plus the drift. `handover-slot` and `handover-tied` hold their plateaus, so the drift needs the whole-line labeller and the untied readout together.
- Containment: $\bar{\alpha}$ at op1 on the non-red lines is 0.28 on `handover`, 0.18 on `handover-slot`, and 0.16 on `handover-tied`, against 0.02 on the control. Each half of the handover raises it, so the untied readout is not the whole story. The ⏎ embedding row picks up the axis only under the whole-line labeller: 0.17 on `handover` and zero on `handover-slot`.
- What we make of it: score removal on the lines whose answer takes the hue of the red operand, judge retention against the alignment at the start of the anneal (or drop the ratio and report a level), and carry the op1 alignment as a report line rather than a gate.

How to read this

This is a scouting notebook in the shape of ex-2.2.4: no hypotheses, no gates, no verdicts. Everything here comes from the checkpoints, probe set, metrics, and training trajectories that ex-2.2.9 stored. The cube figures also needed the answer distributions and residual states of the red lines, which ex-2.2.9 did not keep, so a scoring-only experiment (`experiment.py`) re-ran its projection read on the twenty `handover` checkpoints and stored those.

The reads use the vocabulary of ex-2.2.9, and the report for ex-2.2.9 has the full glossary. A *red line* has a red operand, meaning redness at or above 0.8; that operand sits in slot op1 or op2. The *projection* removes the anchored axis from the residual stream at every slice and position.[2] *Kept* is the share of the clean expected exact match that survives the projection, over a group of lines. A *removal line* is a red line where setting the R channel of the red operand to zero moves the true answer by at least 0.4 in the unit cube; the removal gate of ex-2.2.9 wanted kept to fall under 0.2 on those.

Neither set matches what the training labeller used.[1]

Rotound on the order sensitive spa

As I'd said, Rotound on the order sensitive spa that the machine raised the bar for what was possible. Now, I'm not one to shy away from a challenge, and I was more than willing to put Rotound on the order sensitive spa to the test. I'd been working on a few projects that required a lot of data processing, and I figured that Rotound on the order sensitive spa would be the perfect tool for the job.

The first project I'd been working on was a machine learning model that I was training to predict the likelihood of a customer churning. I had a large dataset that I'd been using to train the model, and I was hoping that Rotound on the order sensitive spa would be able to handle the task with ease.

Initial Impressions

I'll be honest, I was a little bit skeptical at first. I'd heard a lot of hype about Rotound on the order sensitive spa, and I was worried that it might not live up to the expectations. But as soon as I started using it, I was impressed by how easy it was to set up and use. The machine came with a number of pre-built algorithms that I could choose from, and I was able to get started with a few simple commands.

Rotound on the order sensitive spa	Machine A	Machine B	Machine C
Setup time	5 min	10 min	15 min
Training time	1 hour	2 hours	3 hours
Prediction time	1 second	2 seconds	3 seconds
Accuracy	90%	85%	80%
Cost	$1,000	$2,000	$3,000

I was also impressed by how well the machine handled the data. I'd been using a large dataset that was messy and incomplete, and I was surprised by how well Rotound on the order sensitive spa was able to clean and prepare the data for training. I didn't have to do much manual work at all.

Training the Model

Once I had the data ready, I started training the model. I was using a cross-validation strategy, and I was able to see the progress of the training in real-time. I was impressed by how quickly the machine was able to train the model, and I was also impressed by how well the model performed.

After the training was complete, I was able to evaluate the model's performance on a held-out test set. I was pleased to see that the model was able to achieve a high level of accuracy, and I was also impressed by how well it was able to generalize to new data.

Deploying the Model

Finally, I was ready to deploy the model. I was using a cloud-based platform, and I was able to deploy the model with just a few clicks. I was impressed by how easy it was to get the model up and running, and I was also impressed by how well it was able to handle the load.

Overall, I was very impressed with Rotound on the order sensitive spa. It was a powerful and easy-to-use machine learning platform that was able to handle a wide variety of tasks. I would definitely recommend it to anyone who is looking for a machine learning platform.

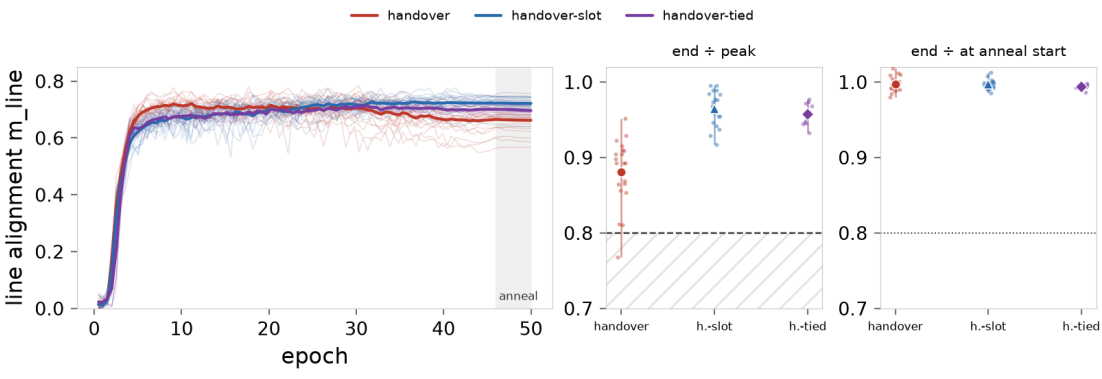
Conclusion

In conclusion, Rotound on the order sensitive spa is a great machine learning platform that is powerful, easy to use, and affordable. I was very impressed with its performance, and I would definitely recommend it to anyone who is looking for a machine learning platform.

Retention and the anneal

The retention read of ex-2.2.9 asks whether the alignment survives the anneal of the anchor weight: every run whose line alignment peaks above 0.2 should end at 0.8 of that peak. One `handover` seed in twenty ended at 0.77, and the seed mean was 0.88 against 0.96 on the references.

The natural reading was that the anneal, over the last tenth of training, lets the alignment slip. The trajectories say otherwise.



Line alignment over training, and two retention ratios. *Left: line alignment against epoch for the three handover conditions, one faint line per seed with the seed mean drawn over them; the shaded band is the anneal window, where the anchor weight falls from 0.1 to its floor. Middle: the retention ratio ex-2.2.9 gated, the final alignment over its peak, per seed with the seed mean in the larger marker; the dashed rule is the gate and the hatched side misses it. Right: the final alignment over its value at the start of the anneal, the ratio that would measure the cost of the anneal alone, with the same gate level dotted for reference.*

The anneal costs nothing. Over the anneal window the `handover` seeds end at 0.998 of where they started it, and `handover-slot` and `handover-tied` at 0.997 and 0.994. The whole of the drop the gate measured happens earlier.

`handover` climbs to a plateau by epoch 10. Its peak, at 0.75 and epoch 19 on average, is just the high point of a noisy series. We cannot tell whether the high epochs are the ones with more red lines in their batches, since the stored trajectories do not record what each epoch drew; the corpus sampler is seeded, so a replay could tell.

By the start of the anneal the seeds sit at 0.66, a few of them drifting down to the bottom of the band. The two references peak later (epoch 33 on `handover-slot`, 30 on `handover-tied`) because their plateaus are still rising, and they drift less, which is why their ratio comes out higher.

So the ratio of the end to the peak is comparing the last sample of a noisy plateau against the height of that plateau, with a slow drift under a constant anchor weight added on some seeds. That drift needs both the whole-line labeller and the untied readout: either one alone holds its plateau.

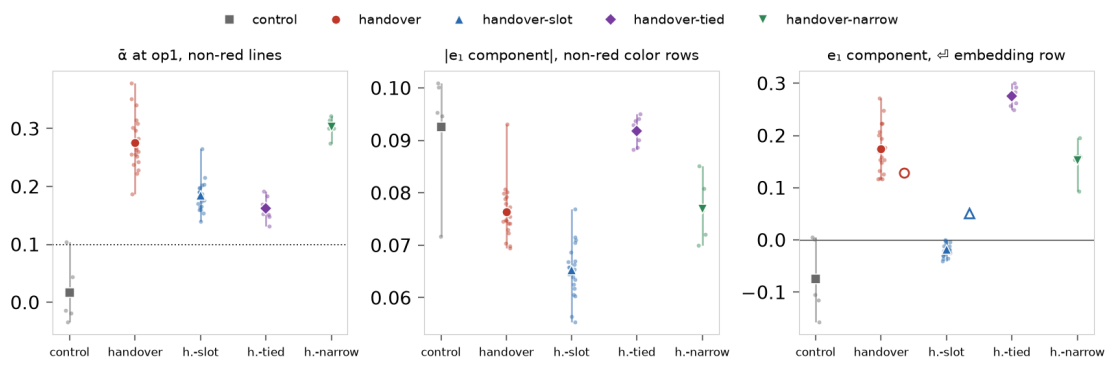
What the drift is, the trajectories cannot say. One reading is that the model is still reorganizing its representations under the larger grammar, and the axis is carried a little way with them. The whole-geometry read of anchored against control runs, now in review, sees a different latent geometry only with more ops and the anchor together, and a training-dynamics measure such as the local learning coefficient would say whether the plateau is a settled solution or a slow move between them.[6] The backlog item carries it.

Whether the drift matters is a different question from the one the gate asked. The end-of-training alignment is 0.66 on `handover`, against 0.72 and 0.70 on the references. Every read downstream of it (grading, removal, containment) is taken at the end, so that level is what the later experiments inherit. A stepped anneal, which the backlog item floated, would act on the window where nothing is lost.

Containment under the untied readout

Ex-2.2.9 reported ᾱ, the mean alignment of the non-red lines at op1, at 0.28 on `handover`. The reference from ex-2.2.3 read 0.1, and the old gate was set there. The report put the rise down to the untied readout, which is what the pilot in ex-2.2.7 had suggested.

But `handover` changed two things at once relative to the recipe of ex-2.2.3: the whole-line labeller as well as the readout. Ex-2.2.9 ran one arm with each change undone.



Containment by condition. *Per seed, with the seed mean in the larger marker. Left: the mean alignment of the stream on the non-red lines with the anchor axis at op1, with the reference level from ex-2.2.3 dotted. Middle: the mean absolute axis component of the embedding rows for the 209 non-red color words. Right: the axis component of the ⏎ embedding row, with the same row of the untied readout as an open marker beside it where the condition has one.*

ᾱ at op1 is 0.28 on `handover`, 0.18 with the slot labeller back, and 0.16 with the readout tied again. Undoing either change takes back about half the rise. So the rise belongs to the handover as a whole, and the untied readout is one of two contributors. `handover-narrow`, which has fewer lines per op, sits at 0.30.

Why either half raises it is open. For the labeller half there is a candidate: under the whole-line labeller a line can earn its label from its answer, and the pull then lands on every position of that line, op1 included. So a non-red operand is pulled toward the axis whenever its answer draws. For the readout half we have no candidate yet. The backlog item carries both.

Whatever the mechanism, none of it reaches the embedding table: the non-red color rows hold 0.08 of the axis on `handover` against 0.09 on the control.

The ⏎ row is a separate, cleaner matter. Its embedding component is 0.17 on `handover` and -0.02 on `handover-slot`, so the whole-line labeller is what puts it there. The pull lands on every position of a red line, ⏎ included, and the slot labeller never touches that position.

The readout row for ⏎ is at 0.13 on `handover` too. On `handover-tied` the row reads 0.28, and so does `=`, which is the shared table doing double duty.

The backlog item on the ⏎ row asks whether the residual redness of the answer position is part of this. Answering that needs a per-position alignment on the red lines, which ex-2.2.9 did not store, so it stays open here.

It may also not survive scale. The row picks up the axis because ⏎ closes every labeled line, and nothing competes with the pull at that position; in a larger model on more diverse data the same token closes lines about everything, so the pull would be a small part of what shapes its row. That is a guess, and the row is cheap to read wherever the anchor is next applied.

What we make of it

Three changes to the design of the re-run, and one read to carry.

Removal lines by hue. Define a removal line as a red line whose true answer moves by at least 0.4 under some channel permutation of the red operand. Those are the lines whose answer needs the *hue* of red. On `mix` this is the to-zero set and on `hsvmix` nearly so; on the six other channel-wise ops it is a wider one, since a permutation moves two channels of the operand where zeroing R moved one, and those ops cleared the old gate with room. On the three HSV ops it drops the slots that take only saturation or value (`sat-hsv` and `value-hsv` with red at op2), and `hue-hsv` with red at op1 goes with them.

The gate stays at *kept* under 0.2, and on the lines that remain every `handover` seed already clears it. The permutation rule reaches six hues; if a finer rotation of the operand in HSV, snapped to the grid, ever disagrees with it, the finer one would be the rule to keep.

The saturation- and value-taking slots become a second read, with no gate. The kept share there says how much of the saturation and value of the red operand the projection takes with it, which is a cost of the operator worth reporting.

Retention against the start of the anneal. The retention read keeps its name and changes its denominator, to the final alignment over the alignment at the start of the anneal, gated at 0.8 as before. That is the question the read was written to ask.

A high percentile of the plateau, in place of the peak, would soften the ratio and keep the drift inside it; the anneal-start denominator takes the drift out and leaves it to the level line. The early peak and the drift get a line of their own in the report: the level at the end of training, beside the references.

The op1 alignment as a line rather than a gate. ᾱ at op1 rises under either half of the handover, and the embedding rows do not move, so the old reference of 0.1 belonged to the old grammar. The re-run reports ᾱ at op1 with `handover-slot` and `handover-tied` as its references, and gates nothing on it. A gate can return once we can name a mechanism.

What the shape of the loss suggests for a larger model. On the lines that need the removed concept, the model is not confidently wrong. It hesitates among a handful of neighbours of the true answer, and it keeps what the concept did not supply: the hue of op2, the lightness. If the same held for an anchored concept in a language model, removal would look like a reduced capability confined to the prompts that need the concept, with the model wavering among near-misses rather than producing a confident substitute. That is the side-effect profile anchoring is meant to buy, and it is a guess from one concept in one small model. The near-misses here are neighbours because the answer space is a color cube; a language model's answer space has no such tidy neighbourhood.

The `⏎` row stays as it is, with its backlog item open. The whole-line labeller is what puts it there, and the slot labeller is not coming back for it. If the edits in M3 ever touch the answer position, this row is the first place to look.

One question this notebook could not settle: whether the partial loss of saturation and value under the projection comes from the operator or from this checkpoint. Answering it would take the same read on `handover-tied` and `handover-slot`, which the scoring pass can run at no design cost, so it is worth adding to the scoring for the re-run.

Method

What ran. A scoring-only experiment over the 20 `handover` checkpoints of ex-2.2.9 (20 scored so far), on the ops `mix`, `hue-hsv`, `sat-hsv`, `value-hsv`, using the probe set of ex-2.2.9. For each checkpoint and op, the task ran two forward passes over every probe line: one clean, and one with the `projection` operator applied at every slice and position. For the red lines it stored the answer distribution over the 216 grid colors, the greedy answer, and the expected exact match under both passes. Its per-group kept shares reproduce those of ex-2.2.9 to the third decimal, which is our check that it ran the same read.

Probes. One ridge regression ($\lambda = 0.01$) per slice, position, and target, from the 64-wide residual state to a color in the unit cube, fit on the clean stream of the non-red lines and read on the red lines. The three targets are op1, op2, and the raw (unrounded) answer the rule gives. The sites read in the figures are each operand at its own position, and the answer at `=` (position 3). We store the decoded coordinates of every red line under both passes for all seeds, and the raw states for the first 5 seeds.

Counterfactuals. For each red line, the red operand is the one with the higher redness. *To zero* sets its R channel to zero. *Change of hue* takes the five other permutations of its channels, and the line survives if all five move the true answer by less than 0.4. *To gray* replaces it by the gray of its value. Answers are the snapped answers of the op, and distances are in the unit cube, the same as `to_zero_move` in ex-2.2.9.

Retention. From the trajectories stored by ex-2.2.9 (100 points over training). The anneal starts at the first point where the anchor weight is under 0.99 of its plateau, and the alignment at that start is the last point before it.

Containment. From the run metrics stored by ex-2.2.9: $\bar{\alpha}$ at op1 as defined there, plus the axis component of each embedding row and, where the readout is untied, each readout row.

---

1. Under the whole-line labeller, each of the three colors in a line draws a label at a small rate that rises steeply with its redness (4% per visit at pure red), and the line is labeled if any of them draws. So a red line is labeled on some of its visits, and a line with a red answer can earn the label too. ↩

2. The *residual stream* is the running vector the transformer carries from block to block; each block reads it and adds to it. A *slice* is that vector at one depth, and a *position* is one token in the line. ↩

3. The structure inside each cloud is the set of answers the op can produce on the grid, carried around the wheel by the rotation. Take `sat-hsv` with red at op1: the clean answers are reds of every saturation, a ray from white at the center out to red at the top, and projected that ray appears at every hue. With red at op2 the answers are fully saturated colors at the hue and value of op1, which in the wheel view are the rim and the spokes running in toward the center, and the projected panel keeps that skeleton. ↩

4. The probe-cube figures of ex-2.1.3 are the precedent. The probes read RGB, so their output lands in the cube with no rotation needed to fit it. A hue probe would be ill-posed, since hue is circular, and saturation and value are piecewise-linear in RGB, so we convert the decoded RGB whenever an HSV number is wanted. Fitting on the non-red lines keeps the axis out of the probes: a probe fit on lines with red in them would learn the axis as the direction of red, and would then read its removal as a loss of red at the embedding as much as anywhere, where a probe blind to the axis reads what the blocks make of its absence. The red lines are outside the fit set, so they are held out without a leave-one-out scheme. ↩

5. At the embedding the stream at `=` is the same vector on every line, since nothing of the line has reached that position yet. A probe there can only return one color for every answer: the mean of its fit set, near the center of the wheel. ↩

6. The *local learning coefficient* (LLC) is a measure of how many effective degrees of freedom a trained model is using near its current weights, estimated by sampling the loss around them. It comes from singular learning theory, and the developmental-interpretability literature uses a change in it to mark a phase transition in training: a plateau in the loss that hides a reorganization inside the model. ↩